CS388 Midterm, Fall 2023 — Worked Solutions

Unofficial solutions with explanations · prepared as a study aid · October 2026

> **NOTE**
> These are my worked solutions, not the official answer key. Where a question is ambiguous I state the assumption I made. Each answer has the reasoning you'd show for partial credit.

Part 1 — Multiple choice / short answer (52 pts)

(1) Which statements about classification are true? (select all)

II, III, IV

- I × — the perceptron converges only if the data are linearly separable.
- II ✓ — logistic regression's NLL is convex in w for fixed features.
- III ✓ — SGD is the standard optimizer for LR.
- IV ✓ — the perceptron is SGD on the loss $\max(0, -y\, w^\top f(x))$.
- V, VI × — on separable data, LR's weights grow toward the max-margin direction, while the perceptron stops at the first separating hyperplane it finds. Their boundaries differ in general, even with infinite epochs.

(2) Text x → one of five labels. Name of the problem?

A. Classification (multiclass).

(3) What parameters can be learned in a deep averaging network?

I, III — feedforward layer weights and word embeddings. A DAN averages embeddings and has no attention, so there's no W^K and no attention map.

(4) Debiasing method of Bolukbasi et al. (2016)?

C — it "neutralizes" gender-neutral words with respect to a gender subspace (removes their projection). It doesn't remove words or shrink dimensionality. D is false because bias remains (Gonen & Goldberg 2019).

(5) Why does the unidirectional RNN predict positive for "...terrible... lackluster... At least the dinner was excellent."?

D — an RNN's final state is heavily influenced by recent tokens, and "excellent" comes last.

(6) Which methods might work better on that example?

I, II, III — bag-of-words, DAN, and Transformer all treat every position without recency bias. There are three negative cues (terrible, lackluster, unconvincing) vs one positive, so order-insensitive aggregation should lean negative. A Transformer can attend to all tokens directly.

(7) P("the frog is blue") under the unsmoothed bigram LM.

D. 1/6
"blue" → UNK. $P = P(\text{the}|\langle s\rangle)\,P(\text{frog}|\text{the})\,P(\text{is}|\text{frog})\,P(\text{UNK}|\text{is}) = 1 \cdot \frac{4}{4} \cdot \frac{2}{2+4} \cdot \frac{1}{1+1} = \frac{1}{6}$.

(8) Which added sentence increases P(the frog is blue) the most?

A. "the frog is black" (black → UNK). Recompute $P(\text{is}|\text{frog}) \cdot P(\text{UNK}|\text{is})$ (the first two factors stay 1 unless "the" gets a new continuation):

Added	New counts	Probability
A	frog→is 3/7; is→UNK 2/3	$\frac{3}{7} \cdot \frac{2}{3} = \frac{2}{7} \approx 0.286$
B	frog→is 5/9; is→UNK 1/4 (is→frog becomes 3)	$\frac{5}{36} \approx 0.139$
C	the→frog 4/5; frog→is 2/6; is→UNK 2/3	$\frac{4}{5} \cdot \frac{1}{3} \cdot \frac{2}{3} = \frac{8}{45} \approx 0.178$
D	frog→is 2/9; is→UNK 1/2	$\frac{1}{9} \approx 0.111$

Original = 0.167, so A gives the largest increase (B and D actually decrease it).

(9) Modified 3-layer Transformer: the middle layer has only MHA (no FFN), and positional encoding is added after layer 1. Parameters vs a standard 3-layer?

C. Fewer — one FFN is missing (≈8d² parameters). Sinusoidal positional encoding has no parameters, so moving it changes nothing.

(10) Aggregate performance vs the standard Transformer?

C. Worse — the first layer's attention has no position information (bag-of-words behavior), and the model has less capacity.

(11) Modified 2-layer Transformer, each layer MHA + 2 FFNs, vs a standard 4-layer?

C. Fewer — same number of FFNs (4 vs 4) but 2 MHA blocks instead of 4, so about 2 × 4d² fewer parameters.

(12) Most likely to improve fine-tuned BERT accuracy?

A — masking more than 15% (moderately more, e.g. ~40%, has been shown to help: Wettig et al., "Should You Mask 15%?"). B removes the bidirectional context that makes BERT strong. C makes pre-training too noisy, and the model never learns to use the [MASK] signal.

(13) Possible shapes for the attention map in the Assignment 3 LM?

II, III — [seq_len, seq_len] (unbatched) or [batch, seq_len, seq_len]. Entry (i,j) = how much position i attends to j. The others are hidden states or weight matrices.

(14) Possible shapes for model outputs fed to the loss?

V — [batch, seq_len, vocab_size]: a distribution over the vocabulary at every position. (Unbatched [seq_len, vocab] is also valid but isn't listed.)

(15) Trained on singular nouns, tested with plurals. What helps?

I, II, III — stemming maps cats → cat; subword tokenization splits cat + s; pretrained embeddings put plurals near singulars. BoW featurization doesn't solve unseen types, and a syntactic parser doesn't fix the lexical mismatch.

(16) Beam search, beam size 2, 2 tokens.

a dog, the dog
Step 1: keep {a (.8), the (.15)}. Step 2: a dog = .8×.98 = .784; a a = a the = .008; the dog = .15×.84 = .126; the a = .0225; the the = .0015. Top 2: **a dog (.784), the dog (.126)**.

(17) How many sequences could sampling return?

9 — all probabilities are nonzero, so 3×3.

(18) Nucleus sampling with p = 0.9?

3
Step 1: sorted a .8 (< .9), + the → .95 ≥ .9 ⇒ nucleus {a, the}.
After a: dog .98 ≥ .9 ⇒ {dog}. After the: dog .84 < .9, + a → .99 ⇒ {dog, a}.
Sequences: a dog, the dog, the a.

(19) Viterbi runtime, n words and k tags.

O(nk²) — at each of n positions, for each of k tags, maximize over k previous tags.

(20) How many paths does Viterbi search over?

O(kⁿ) — it implicitly considers all tag sequences.

(21) Each tag can be followed by at most l tags. How many paths?

O(k · l^(n−1)) — k choices for the first tag, then at most l for each of the remaining n−1 (often written $O(k\,l^n)$).

Part 2 — Long answer (48 pts)

Q2 (18 pts) Vocabulary [good, great, not]; data: goodx (+), great (+), not greatq (−), not good (−).

(a) Typos make tokens out of vocabulary (goodx, greatq contribute nothing):
[0,0,0], [0,1,0], [0,0,1], [1,0,1]

(b) Perceptron, w = 0, score ≥ 0 → positive.

Ex	f(x)	score	pred	gold	w after
1	[0,0,0]	0	+	+	[0,0,0]
2	[0,1,0]	0	+	+	[0,0,0]
3	[0,0,1]	0	+	−	[0,0,−1]
4	[1,0,1]	−1	−	−	[0,0,−1]

Final w = [0, 0, −1]

(c) Greedy (longest-match, left-to-right) segmentation: goodx → *good x*; great → *great*; not greatq → *not great q*; not good → *not good*.
New BoW vocabulary (in the given order): **[good, x, q, not, gre, at, great]**
Features: **[1,1,0,0,0,0,0], [0,0,0,0,0,0,1], [0,0,1,1,0,0,1], [1,0,0,1,0,0,0]**
(Equivalently, drop the unused pieces gre and at: [good, x, q, not, great].)

(d) greqat → **gre q at** (no piece starting with "greq" exists, so greedy takes gre, then q, then at).

(e) Edit-distance repair (2): a typo anywhere in a word maps back to the nearest real word (greqat → great), while subword tokenization only helps when the typo is at a word edge. An internal typo shatters the word into meaningless pieces (gre/q/at).

Q3 (13 pts) Word vectors v_1=(0,0), v_2=(1,1), v_3=(−1,1); contexts c_1=(0,0), c_2=(0,1), c_3=(1,0); take e = 3. Skip-gram: $P(c \mid w) \propto \exp(v_w \cdot c)$.

(a) $v_1 \cdot c_j = 0$ for all j ⇒ **(1/3, 1/3, 1/3)**
(b) $v_2 \cdot c$ = (0, 1, 1) ⇒ exp = (1, 3, 3) ⇒ **(1/7, 3/7, 3/7)**
(c) 1 extra dimension. Words 4 and 5 form a separate cluster that only co-occurs internally, so a single new axis can carry their mutual association while staying orthogonal to the existing two dimensions.
(d) One valid construction (third dimension = new):
$v_4 = v_5 = (0, 0, 10), \quad c_4 = c_5 = (0, -10, 10).$

- 4/5 relationship: $v_4 \cdot c_4 = v_4 \cdot c_5 = 100$ (huge), while $v_4 \cdot c_{1,2,3} = 0$ ⇒ $P(\cdot \mid 4) \approx (0, 0, 0, \frac{1}{2}, \frac{1}{2})$, and the same for word 5. ✓
- Original words: $v_2 \cdot c_{4,5} = -10$ and $v_3 \cdot c_{4,5} = -10$ ⇒ $e^{-10} \approx 0$, so their distributions over 1–3 are unchanged. ✓
- Caveat: $v_1 = (0, 0, 0)$ has dot product 0 with every context, so no choice of c_4, c_5 can keep v_1's mass off words 4 and 5. Mentioning this shows understanding.

Q4 (17 pts) Trees: (S (NP (PRP I)) (VP (VBP teach) (NP (NNS students)) (PP (IN about) (NP (NNP NLP))))) and (S (NP (PRP he)) (VP (VBP gives) (NP (NNS lessons)))).

(a) Grammar (no lexicon, no binarization):

```
S  → NP VP
NP → PRP  |  NNS  |  NNP
VP → VBP NP PP  |  VBP NP
PP → IN NP
```

(b) "him teach I" (assuming *him* is tagged PRP like I/he): NP→PRP covers him; NP→PRP covers I; VP→VBP NP covers "teach I"; S→NP VP.
(S (NP (PRP him)) (VP (VBP teach) (NP (PRP I)))) — grammatical in structure even though the case is wrong. A plain CFG doesn't capture case agreement.

(c) Vertical Markovization on NPs only:

```
S     → NP^S VP
NP^S  → PRP
NP^VP → NNS
NP^PP → NNP
VP    → VBP NP^VP PP  |  VBP NP^VP
PP    → IN NP^PP
```

(d) Not parseable. The object NP under VP must be NP^VP → NNS, but "I" is PRP. Only subject NPs (NP^S) can be pronouns.
(e) Subset. Parent annotation splits symbols, so every tree of the new grammar maps to a tree of the old grammar (strip the annotations), but not vice versa.